

HealthScope — Sistem Dokümantasyonu

Kan tahlili → klinik çıkarım → biyo-nutrisyonel protokol

Bu belge uygulamanın uçtan uca nasıl çalıştığını anlatır: verinin hangi katmanlardan geçtiğini, her katmanın hangi kararı verdiğini ve bu kararların hangi ölçümle doğrulandığını.

HealthScope teşhis koymaz. Karar *destek* sistemidir; çıktısı laboratuvar değerlerinin istatistiksel yorumudur ve hekim değerlendirmesinin yerine geçmez.

1. Temel tasarım kararı

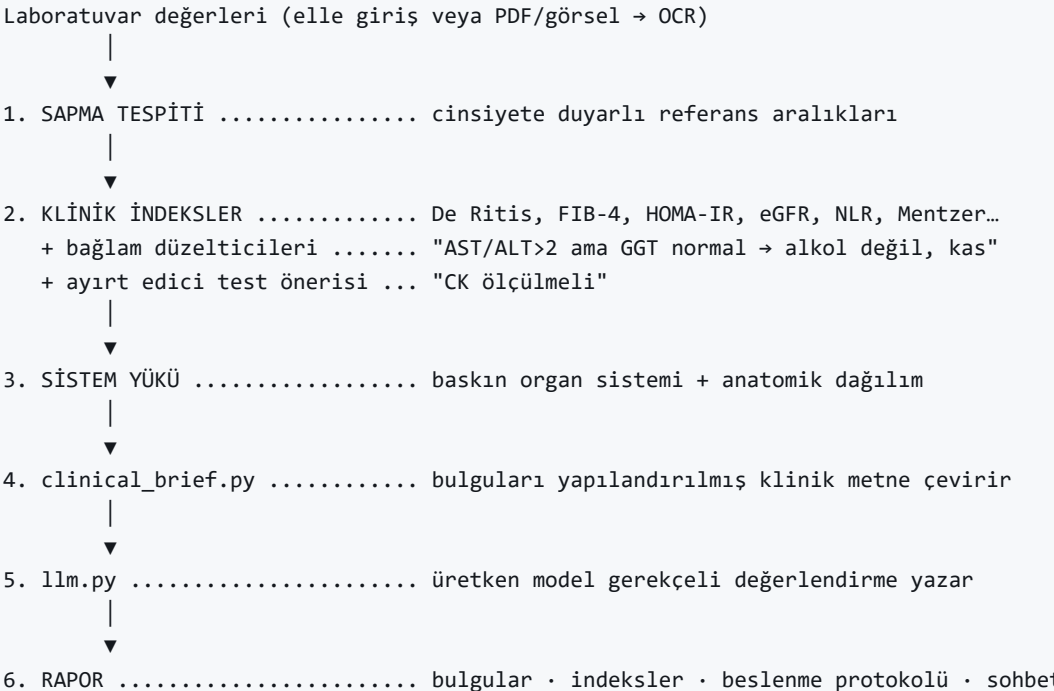
Sistemin merkezinde tek bir mimari tercih var:

Deterministik kural motoru sonucu üretir; dil modeli onun _üzerinde_ akıl yürütür — tersi değil.

Bu tercih keyfi değil, ölçüm sonucu. Ham laboratuvar değerleri doğrudan dil modeline verildiğinde model ağır karaciğer, ağır böbrek ve ağır anemi vakalarına birbirine çok benzeyen cevaplar üretiyordu: sayı dizisi, dil modeli için ayırt edici bir sinyal değil. Aynı vakalarda kural motoru — bir bölme işlemiyle — alkolik ve viral hepatiti ayırabiliyor.

Dolayısıyla dil modeli **sınıflandırıcı** olarak değil, **açıklayıcı** olarak konumlandırıldı. Bulguları kural motoru bulur; model bu bulguları klinik bir gerekçeye bağlar ve hastanın anlayacağı dile çevirir.

2. Uçtan uca akış



1–4 arası **tamamen deterministik**: aynı girdi her zaman aynı çıktıyı verir, test edilebilir, açıklanabilir. 5. adım olasılıksaldır ve **opsiyoneldir** — kapatıldığında sistem çalışmaya devam eder, yalnızca gerekçeli anlatı üretilmez.

3. Veri modeli: tek doğruluk kaynağı

Laboratuvar parametreleriyle ilgili her şey tek bir dosyada, `database.json` içindeki `PARAMETER_CATALOG` bölümünde tanımlıdır:

```
"insulin": {
  "label": "İnsülin (Açlık)",
  "unit": "uIU/mL",
  "domain": "Endokrinoloji",
  "group": "diyabet",
  "ref": { "min": 2.6, "max": 24.9 },
  "ocr": ["\\b[Iİ]NS[UÜ]L[Iİ]N\\b"],
  "nutrition": { "high": "insulin_resistance" }
}
```

Backend bunu `catalog.py`, arayüz `src/lib/catalog.ts` üzerinden okur — **aynı dosyayı**. Form alanları, referans aralıkları, OCR desenleri ve beslenme eşleşmesi elle senkronize edilmez.

Bunun pratik sonucu: "formda alan var ama backend tanımıyor" ya da "protokol var ama anahtar tutmuyor" sınıfı hatalar yapısal olarak oluşmaz. Yeni bir parametre eklemek için tek yapılacak katalogda bir kayıt açmaktır; form alanı, OCR desteği ve sapma tespiti otomatik gelir. `catalog.validate()` her açılıшта, `tests/test_catalog.py` her testte bu tutarlılığı doğrular.

Envanter

Bileşen	Adet
Laboratuvar parametresi	60
Parametre grubu	6
Klinik indeks formülü	16
Beslenme protokolü	39
Semptom protokolü	4
Klinik terim sözlüğü	340
Sentetik test vakası	130

4. Sapma tespiti

Her parametre için referans aralığı karşılaştırması yapılır. İki nokta önemli:

Cinsiyete duyarlılık. Hemoglobin, hematokrit, ferritin, kreatinin ve demir gibi parametrelerin referans aralıkları cinsiyete göre değişir. Katalog `ref_male` / `ref_female` alanlarını taşır; tek bir "ortalama" aralık kullanmak kadın hastalarda yanlış negatif, erkeklerde yanlış pozitif üretti.

Şiddet, yüzde değil oran. Bir değerin ne kadar "kötü" olduğu, referans sınırından uzaklığının **aralık genişliğine oranıyla** ölçülür:

$$\text{şiddet} = (\text{sınır dışı mesafe}) / (\text{referans aralığı genişliği})$$

Yüzde sapma kullanmak dar aralıklı parametreleri (ör. potasyum) sistematik olarak hafif, geniş aralıklıları (ör. trigliserid) sistematik olarak ağır gösterirdi.

Sistem yükü hesabı

Bulgular klinik alanlara dağıtılır ve her alan puanlanır:

$$\text{alan puanı} = \sum \sqrt{\text{şiddet}} \times (1 + 0.20 \times (\text{bulgu sayısı} - 1))$$

Karekök sıkıştırma tek bir aşırı değerin tüm tabloyu ele geçirmesini önler; sayı çarpanı ise "üç orta bulgu, bir ağır bulgudan daha anlamlıdır" sezgisini kodlar. İki alan birbirine yakın puan aldığında (%85 eşiği) ikisi de baskın kabul edilir — tek bir alan seçmeye zorlamak çok sistemli tabloları gizlerdi.

5. Klinik indeksler

Tek tek parametrelere bakmak, birden fazla parametrenin **oranında** saklı bilgiyi kaçıır. 16 literatür formülü deterministik olarak hesaplanır:

İndeks	Ne ayırır
De Ritis (AST/ALT)	alkolik ↔ viral hepatit
Mentzer (MCV/RBC)	talasemi ↔ demir eksikliği
Transferrin satürasyonu	demir eksikliği ↔ demir yüklenmesi
BUN/Kreatinin	prerenal ↔ intrinsek renal azotemi
FIB-4	karaciğer fibrozis riski
HOMA-IR	insülin direnci
eGFR (CKD-EPI 2021)	kronik böbrek hastalığı evresi
NLR, PLR	sistemik inflamatuvar yük
TG/HDL, Non-HDL	aterojenik risk
Düzeltilmiş kalsiyum (Payne)	albümin düşükken gerçek kalsiyum
Lipaz/Amilaz	pankreatit etiyolojisi

Formüller `indices.py` içinde kod olarak durur; eşikler, yorum metinleri ve kaynakça `database.json` 'da. JSON'dan formül `eval` etmek hem güvensiz hem test edilemez olurdu.

Uygulanabilirlik koşulu. Bazı indeksler yalnızca belirli bir tablo varken anlamlıdır. Mentzer indeksi mikrositoz yoksa hesaplanmaz — aksi hâlde sağlıklı bir hastada "demir eksikliği lehine" gibi yanıltıcı bir yorum üretti. Bu, `applicable_when` alanıyla tanımlanır.

Bağlam düzelticileri. Bir oranın yorumu, tabloda başka ne olduğuna göre değişir. AST/ALT > 2 klasik olarak alkolik karaciğer hastalığını düşündürür; ama GGT normalse alkol açıklaması zayıflar ve kas kaynaklı AST artışı öne çıkar. Sistem bu durumda yorumu değiştirir ve **ayırt edici test önerir**: "CK ölçülmeli."

Bu davranış gerçek bir hasta raporunda doğrulandı: sistem "kas kaynaklı AST artışı" yorumuna ve CK önerisine ulaştı — hastayı gören hekimin izlediği yolun aynısı.

6. Anatomik dağılım

Bulgular organ sistemlerine dağıtılır ve önden görünüm bir şemada işaretlenir. Eşleme iki katmanlıdır:

1. **Alan bazlı**: bir klinik alan ilgili tüm organlara katkı verir.
2. **Parametre bazlı ezme**: alan eşlemesi tek başına yanıltıcı olabiliyor —

"Endokrinoloji" hem tiroidi hem pankreası kapsadığı için yüksek insülin tiroidi de boyuyordu. Bu parametreler doğrudan ilgili organa bağlanır.

Şema `src/lib/anatomy.ts` (eşleme + şiddet mantığı) ve `AnatomyFigure.tsx` (çizim) olarak ayrılmıştır; görselleştirme değişse bile klinik eşleme aynı kalır.

7. Klinik özet ve üretken katman

`clinical_brief.py` bulguları yapılandırılmış Türkçe bir metne çevirir: hangi parametreler ne kadar sapmış, hangi indeksler hesaplanmış, hangi hipotez elenmiş. **Elenen hipotezin de yazılması kasıtlıdır** — model yanlış yöne sapmasın diye.

Bu metin `llm.py` üzerinden üretken modele verilir. Model yeni bulgu üretmez; verilen öznelliklere dayalı gerekçeli bir değerlendirme yazar.

Prompt'a giren serbest metinler (kronik hastalık, genetik risk) önce temizlenir; sağlık verisi işlendiği için CORS'ta joker origin kullanılmaz ve yükleme boyutu sınırlıdır.

Sohbet katmanı

Hasta, üretilen değerlendirme üzerinde soru sorabilir. Sohbet bağlamı yine kural motorunun çıktısından kurulur — model konuşurken de aynı deterministik zeminde kalır.

8. OCR yolu

PDF veya görsel tahlil raporu yüklendiğinde EasyOCR (TR + EN) metni çıkarır ve katalogdaki OCR desenleriyle parametre değerleri eşleştirilir.

Şüpheli değer tespiti. OCR ondalık ayracı kaybedebiliyor: `2.9` değeri `29` olarak okunabiliyor. Sistem, okunan değeri referans aralığıyla karşılaştırıp ondalık kayması olasılığını tespit eder ve arayüzde tek tıkla düzeltme sunar. Sessizce kabul etmek, sonraki tüm katmanları kirlletirdi.

9. Ölçüm sonuçları

Ölçüm, `presets.json` içindeki **130 sentetik klinik vaka** üzerinde yapılır. Vakalar elle yazılmaz; `scripts/build_presets.py` içindeki klinik arketiplerden üretilir (42 arketip $\times$ 3 şiddet + 4 negatif kontrol). Her arketip hastalığın tipik panelini tanımlar; hafif ve ağır varyantlar referans sınırından sapma miktarı ölçeklenerek türetilir.

Deterministik katmanlar

Katman	Sonuç
Sapma tespiti	1082 / 1082 (%100)
Beslenme protokolü eşleşmesi	812 / 812 (%100)
Baskın alan tespiti	108 / 130 (%83)
Klinik indeks isabeti	81 / 127 (%64)

Sapma tespiti ve beslenme eşleşmesinin %100 olması beklenen sonuçtur: ikisi de katalogdan doğrudan türeyen deterministik işlemlerdir; buradaki değer, ölçümün **katalog bütünlüğünü** doğrulamasıdır.

Baskın alan tespitindeki 22 kaçırılan vakanın önemli bölümü tartışmalı etiketlerdir (hemokromatoz, non-alkolik yağlı karaciğer, gut): bu tablolarda "doğru" alan tek değildir ve sistem çoğu zaman ikincil olarak doğru alanı da işaretlemektedir.

Çıkarım katmanları karşılaştırması

Beklenen klinik konusu tanımlı 126 vakada üç yaklaşım aynı ölçütle karşılaştırıldı: üretilen metnin, vakanın beklenen klinik konularıyla örtüşmesi.

Yaklaşım	İsabet	Ortalama süre
BERTurk fill-mask (tek başına)	82 / 126 (%65)	27 ms
Kural motoru + klinik indeks	126 / 126 (%100)	< 1 ms
Hibrit: kural motoru $\rightarrow$ üretken model	119 / 126 (%94)	14.4 s

Bu tabloyu doğru okumak. Üç satır aynı şeyi ölçmüyor:

- Kural motoru satırındaki %100 tautolojiktir.** Ölçüt, motorun ürettiği

bulgu etiketlerini ve indeks yorumlarını tarıyor; beklenen konu zaten büyük ölçüde o etiketlerin içinde. Bu satır "kural motoru en iyisi" demez, ölçütün üst sınırını gösterir.

- Anlamlı karşılaştırma birinci ile üçüncü satır arasındadır:** aynı üretken

metin ölçütünde tek başına dil modeli %65'te kalırken, kural motorunun çıkardığı klinik özneliliklerle beslendiğinde %94'e çıkıyor. Akıl yürütmenin yarısı zaten yapılmış hâlde geliyor.

- Süre farkı 500 kata yakındır.** Hibrit katmanın maliyeti gerçektir; bu

yüzden opsiyoneldir ve kapalıyken deterministik katmanlar tam çalışır.

Ölçüt cömerttir (alt dizge araması yapar), dolayısıyla rakamlar bir **üst sınır** olarak okunmalıdır. Ölçüm ayrıca **sentetik** vakalar üzerindedir; gerçek hasta verisiyle doğrulama ayrı bir çalışmadır.

Yeniden üretmek için:

```
python scripts/benchmark_inference.py --url http://127.0.0.1:8000
```

10. Sınırlar

- Sistem **teşhis koymaz**; olasılıksal bir değerlendirme üretir.
- Vaka havuzu **sentetiktir**. Gerçek hasta verisiyle doğrulama ayrı ve daha

zorlu bir çalışmadır; şu ana kadar tek tek gerçek raporlarla yapılan kontroller referans aralığı ve klinik bağlam hatalarını ortaya çıkarmış ve düzeltilmiştir.

- Üretken katman opsiyoneldir ve donanım gerektirir; kapalıyken deterministik

katmanlar tam olarak çalışır.

- OCR, düzensiz tarama kalitesinde hata yapabilir; şüpheli değer tespiti bunu

azaltır ama ortadan kaldırmaz.

11. Teknoloji

Katman	Teknoloji
Arayüz	Next.js 16 (App Router, statik export), React 19, Tailwind v4, Recharts
Sunucu	Python 3.12+, FastAPI, Pydantic v2
Örüntü çıkarımı	BERTurk — dbmdz/bert-base-turkish-cased temelli, bu proje için tıbbi metinlerle fine-tune edildi (Ruhadam2020/checkpoint-85239)
Üretken katman	Qwen2.5-3B-Instruct (fp16, ~5.8 GB VRAM) — opsiyonel
OCR	EasyOCR (TR + EN), PDF için pdf2image + Poppler
Veri	database.json — tek dosya, tek doğruluk kaynağı